APS502H1

Financial Engineering

Fall 2024

Computational Project Report

Present to

Professor Roy Kwon

By

Nara Cheyklin

ID: 1011835971

Problem 1 Part 1(a)

<u>Goal</u>: Minimize the total cost of the bond portfolio

Minimize $c' x$

<u>Constraint:</u> 1. The obligation of required cash flow in each year

$Ax >= b$

2. Each number of bonds must not be less than 0

$lb = zeros(12,1)$

<u>Variable:</u>

Let's c be the bond price vector $\begin{bmatrix} 109 \\ 94.8 \\ 99.5 \\ 93.1 \\ 97.2 \\ 96.3 \\ 92.9 \\ 110 \\ 104 \\ 101 \\ 107 \\ 102 \end{bmatrix}$ Let's x be the number of bond vector = $\begin{bmatrix} units\ of\ bond\ 1 \\ units\ of\ bond\ 2 \\ units\ of\ bond\ 3 \\ units\ of\ bond\ 4 \\ units\ of\ bond\ 5 \\ units\ of\ bond\ 6 \\ units\ of\ bond\ 7 \\ units\ of\ bond\ 8 \\ units\ of\ bond\ 9 \\ units\ of\ bond\ 10 \\ units\ of\ bond\ 11 \\ units\ of\ bond\ 12 \end{bmatrix}$

Let's A be the cash flow matrix= $\begin{bmatrix} 10 & 7 & 8 & 6 & 7 & 6 & 5 & 10 & 8 & 6 & 10 & 7 \\ 10 & 7 & 8 & 6 & 7 & 6 & 5 & 10 & 8 & 6 & 110 & 107 \\ 10 & 7 & 8 & 6 & 7 & 6 & 5 & 110 & 108 & 106 & 0 & 0 \\ 10 & 7 & 8 & 6 & 7 & 106 & 105 & 0 & 0 & 0 & 0 & 0 \\ 10 & 7 & 8 & 106 & 107 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 110 & 107 & 108 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

Let's b be the obligation cash flow vector = $\begin{bmatrix} 500 \\ 200 \\ 800 \\ 200 \\ 800 \\ 1200 \end{bmatrix}$

Let's lb be the lower bound number of each bond zeros(12,1)

```
>> %Price Vector
>> c

c =

  109.0000
   94.8000
   99.5000
   93.1000
   97.2000
   96.3000
   92.9000
  110.0000
  104.0000
  101.0000
  107.0000
  102.0000

>> %Cash Flow Matrix
>> A

A =

    10     7     8     6     7     6     5    10     8     6    10     7
    10     7     8     6     7     6     5    10     8     6   110   107
    10     7     8     6     7     6     5   110   108   106     0     0
    10     7     8     6     7   106   105     0     0     0     0     0
    10     7     8   106   107     0     0     0     0     0     0     0
   110   107   108     0     0     0     0     0     0     0     0     0

>> %Obligation Vector
>> b

b =

         500
         200
         800
         200
         800
        1200

>> %Lower Bound
>> lb

lb =

     0
     0
     0
```

```
     0
     0
     0
     0
     0
     0
     0
     0
     0

>> %Find the optimal number of each bond
>> [x, fval]= linprog(c,-A,-b,[],[],lb,[])

Optimal solution found.


x =

   44.6900
         0
         0
         0
    3.3000
         0
         0
    3.0000
         0
         0
         0
         0


fval =

   5.5220e+03

>> %x is a vector of a number of each bond
>> %fval is the total price of the bond portfolio
>> %There are 44.69 units of bond 1, 3.3 units of bond 5, and 3 units of bond 8
>> %The total cost of the portfolio is 5522
>>
```

Problem 1 Part 1 (b)

<u>Goal</u>: Minimize the total cost of the bond portfolio

Minimize c_new' x

<u>Constraint:</u> 1. The obligation of required cash flow in each year

Ax>= b

2. Each number of bonds must not be less than 0

lb = zeros(12,1)

3. Bond 2 and 3 must not be more than 5% of the total units

Units of bond 2<= 0.05total units and Units of bond 3 <= 0.05total units

<u>Additional Variables from Problem 1 part 1(a):</u>

Let's A_add1 be additional row constraint for bond 2 and bond 3 and total units=

$$\begin{bmatrix} 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.05 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.05 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & 1 \end{bmatrix}$$

Let's A_add2 be additional column constraint for bond 2 and bond 3 and total units = zeros(6,1)

Let's b_add be additional bound = [0;0;0]

Let's c_new be c with additional 0 in the last row

$$\text{x_new}=\begin{bmatrix} units\ of\ bond\ 1 \\ units\ of\ bond\ 2 \\ units\ of\ bond\ 3 \\ units\ of\ bond\ 4 \\ units\ of\ bond\ 5 \\ units\ of\ bond\ 6 \\ units\ of\ bond\ 7 \\ units\ of\ bond\ 8 \\ units\ of\ bond\ 9 \\ units\ of\ bond\ 10 \\ units\ of\ bond\ 11 \\ units\ of\ bond\ 12 \\ total\ units \end{bmatrix}$$

<u>Methodology:</u>

with MATLAB,

[x_new=the optimal number-of-bond vector, fval_new = the total cost of portfolio of the optimal solution]

= *linprog(c_new',-[A, A_add2; A_add1],-[b;b_add],[],[],lb,[])* ,then I obtained

```
>> %Additional variables
>> A_add1

A_add1 =

  Columns 1 through 9

         0   -1.0000         0         0         0         0         0         0 ↙
0
         0         0   -1.0000         0         0         0         0         0 ↙
0
   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000 ↙
-1.0000

  Columns 10 through 13

         0         0         0    0.0500
         0         0         0    0.0500
   -1.0000   -1.0000   -1.0000    1.0000

>> A_add2

A_add2 =

     0
     0
     0
     0
     0
     0

>> b_add

b_add =

     0
     0
     0

>> %Combining with A and b from (a)
>> A_new

A_new =

  Columns 1 through 9

   10.0000    7.0000    8.0000    6.0000    7.0000    6.0000    5.0000   10.0000 ↙
8.0000
   10.0000    7.0000    8.0000    6.0000    7.0000    6.0000    5.0000   10.0000 ↙
8.0000
```

```
   10.0000    7.0000    8.0000    6.0000    7.0000    6.0000    5.0000  110.0000 ↙
108.0000
   10.0000    7.0000    8.0000    6.0000    7.0000  106.0000  105.0000         0 ↙
0
   10.0000    7.0000    8.0000  106.0000  107.0000         0         0         0 ↙
0
  110.0000  107.0000  108.0000         0         0         0         0         0 ↙
0
         0   -1.0000         0         0         0         0         0         0 ↙
0
         0         0   -1.0000         0         0         0         0         0 ↙
0
   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000   -1.0000 ↙
-1.0000

  Columns 10 through 13

    6.0000   10.0000    7.0000         0
    6.0000  110.0000  107.0000         0
  106.0000         0         0         0
         0         0         0         0
         0         0         0         0
         0         0         0         0
         0         0         0    0.0500
         0         0         0    0.0500
   -1.0000   -1.0000   -1.0000    1.0000

>> b_new

b_new =

         500
         200
         800
         200
         800
        1200
           0
           0
           0

>> c_new

c_new =

  109.0000
   94.8000
   99.5000
   93.1000
   97.2000
```

```
   96.3000
   92.9000
  110.0000
  104.0000
  101.0000
  107.0000
  102.0000
         0

>> %Finding Optimal Solution
>> [x_new, fval_new]=linprog(c_new', -A_new,-b_new,[],[],lb,[])
Warning: Length of lower bounds is < length(x); filling in missing lower bounds with - ↙
Inf.
> In checkbounds (line 32)
In linprog (line 271)

Optimal solution found.


x_new =

   44.6900
         0
         0
         0
    3.3000
         0
         0
    3.0000
         0
         0
         0
         0
   50.9900


fval_new =

   5.5220e+03

>>
```

Problem 1 Part 2

Goal: Minimize the total cost of the bond portfolio

Minimize c'x

Constraint: 1. Each number of bonds must not be less than 0

lb = zeros(12,1)

2. The sum of (units of each bond) (its price) must match the present value of liabilities

3. The sum of (units of each bond) (each bond duration) must match the duration of liability

Additional Variables from Part 1:

Let Aeq be [[present value of each bond] ; [durations of each bond]] =

[[108.99544119416079, 94.7974149528402, 99.53009036661373, 93.10190581446297, 97.21789435767225, 96.36989564784616, 92.93796685018896, 109.85370079299102, 104.50540965491467, 99.15711851683834, 107.53071510623309, 102.00164575219686]

; [484.81772630028456, 441.7837348255197, 456.12839865044134, 382.60293221465054, 393.53388515906147, 329.37238347580234, 321.61160267687325, 283.75420557859707, 273.8887979543835, 264.0233903301699, 194.0936548208736, 186.3283584082791]]

Let beq be [present value of liabilities; duration of liability] =

[2757.9367548911464; 9863.211578452068]

Methodology:

With Python, I obtained the present value of liabilities, the duration of liabilities and the durations of bonds and input them into MATLAB.

With MATLAB,

[x=the optimal number-of-bond vector, fval = the total cost of portfolio of the optimal solution]

= *linprog(c',[],[],Aeq,beq,lb,[])* ,then I obtained

```
>> %Problem 1 Part 2
>> Bond_NPV

Bond_NPV =

  Columns 1 through 9

  108.9954   94.7974   99.5301   93.1019   97.2179   96.3699   92.9380  109.8537 ↙
104.5054

  Columns 10 through 12

   99.1571  107.5307  102.0016

>> Bond_duration

Bond_duration =

  Columns 1 through 9

  484.8177  441.7837  456.1284  382.6029  393.5339  329.3724  321.6116  283.7542 ↙
273.8888

  Columns 10 through 12

  264.0234  194.0937  186.3284

>> Aeq=[Bond_NPV;Bond_duration]

Aeq =

  Columns 1 through 9

  108.9954   94.7974   99.5301   93.1019   97.2179   96.3699   92.9380  109.8537 ↙
104.5054
  484.8177  441.7837  456.1284  382.6029  393.5339  329.3724  321.6116  283.7542 ↙
273.8888

  Columns 10 through 12

   99.1571  107.5307  102.0016
  264.0234  194.0937  186.3284

>> beq

beq =

   1.0e+03 *

    2.7579
```

```
    9.8632

>> %Find the optimal solution
>> [x,fval]=linprog(c',[],[],Aeq, beq, lb,[])

Optimal solution found.


x =

         0
         0
   13.4945
         0
         0
         0
         0
         0
   13.5384
         0
         0
         0


fval =

   2.7507e+03

>>
```

Problem 2 Part 2

	CME	BR	CBOE	ICE	ACN
Arithmetic Expected Return	0.01345	0.01667	0.01454	0.01326	0.01497
Geometric Expected Return	0.01189	0.01460	0.01259	0.01146	0.01269
Standard Deviation	0.05657	0.06507	0.06223	0.06080	0.06775

Covariance Table:

	SPY	GOVT	EEMV	CME	BR	CBOE	ICE	ACN
SPY	0.00205	0.00012	0.00115	0.00090	0.00191	0.00104	0.00086	0.00233
GOVT	0.00012	0.00022	0.00014	0.000003	0.00023	0.00001	0.00005	0.00022
EEMV	0.00115	0.00014	0.00130	0.00041	0.00094	0.00047	0.00032	0.00117
CME	0.00090	0.000003	0.00041	0.00320	0.00112	0.00189	0.00117	0.00112
BR	0.00191	0.00023	0.00094	0.00112	0.00423	0.00123	0.00070	0.00288
CBOE	0.00104	0.00001	0.00047	0.00189	0.00123	0.00387	0.00151	0.00156
ICE	0.00086	0.00005	0.00032	0.00117	0.00070	0.00151	0.00370	0.00107
ACN	0.00233	0.00022	0.00117	0.00112	0.00288	0.00156	0.00107	0.00459

Goal: Minimize the variance of ETFs

Minimize $0.5x' Q x$

Constraint: 1. The expected return of portfolio must not be less than the expected goal return

$$Ax>=R$$

2. The sum of proportions of funds in a portfolio must be equal to 1

$$Aeq\ x= beq$$

Variables:

Let's Q be the covariance matrix of portfolio =
[0.00205,0.00012,0.00115,0.00090,0.00191,0.00104,0.00086,0.00233;
0.00012,0.00022,0.00014,0.000003,0.00023,0.00001,0.00005,0.00022;

0.00115,0.00014,0.00130,0.00041,0.00094,0.00047,0.00032,0.00117;

0.00090,0.000003,0.00041,0.00320,0.00112,0.00189,0.00117,0.00112;

0.00191,0.00023,0.00094,0.00112,0.00423,0.00123,0.00070,0.00288;

0.00104,0.00001,0.00047,0.00189,0.00123,0.00387,0.00151,0.00156;

0.00086,0.00005,0.00032,0.00117,0.00070,0.00151,0.00370,0.00107;

0.00233,0.00022,0.00117,0.00112,0.00288,0.00156,0.00107,0.00459]

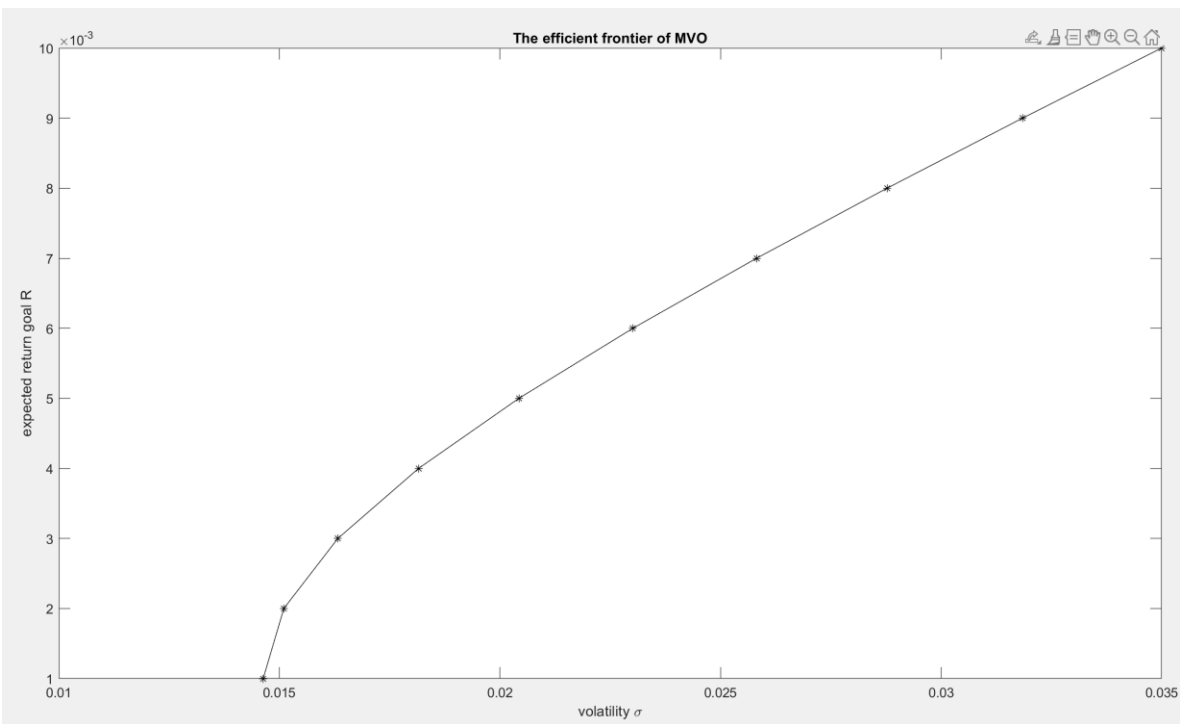
Let's A be - the geometric expect rate of return =

[0.01241, 0.00045, 0.00437, 0.01189, 0.01460, 0.01259, 0.01146, 0.01269]

Summary:

These are the weight of each fund and portfolio volatility and variance for each expected goal return.

Expected Goal Return R	Weight of SPY	Weight of GOVT	Weight of EEMV	Portfolio Volatility	Portfolio Variance
0.10%	0.0342	0.9293	0.0366	0.0146	0.000213
0.20%	0.1343	0.8800	-0.0142	0.0151	0.000228
0.30%	0.2346	0.8306	-0.0651	0.0163	0.000266
0.40%	0.3349	0.7812	-0.1160	0.0182	0.000331
0.50%	0.4351	0.7318	-0.1669	0.0204	0.000416
0.60%	0.5354	0.6824	-0.2178	0.0230	0.000529
0.70%	0.6357	0.6330	-0.2687	0.0258	0.000666
0.80%	0.7360	0.5836	-0.3196	0.0288	0.000829
0.90%	0.8363	0.5342	-0.3705	0.0318	0.001011
1.00%	0.9366	0.4848	-0.4214	0.0350	0.001223



Efficient Frontier

```
>> %Problem 2 part 2
>> n=8

n =

     8

>> mu

mu =

    0.0124    0.0004    0.0044    0.0119    0.0146    0.0126    0.0115    0.0127

>> goal_R

goal_R =

  Columns 1 through 9

    0.0010    0.0020    0.0030    0.0040    0.0050    0.0060    0.0070    0.0080 ↙
0.0090

  Column 10

    0.0100

>> Q

Q =

    0.0021    0.0001    0.0011    0.0009    0.0019    0.0010    0.0009    0.0023
    0.0001    0.0002    0.0001    0.0000    0.0002    0.0000    0.0001    0.0002
    0.0011    0.0001    0.0013    0.0004    0.0009    0.0005    0.0003    0.0012
    0.0009    0.0000    0.0004    0.0032    0.0011    0.0019    0.0012    0.0011
    0.0019    0.0002    0.0009    0.0011    0.0042    0.0012    0.0007    0.0029
    0.0010    0.0000    0.0005    0.0019    0.0012    0.0039    0.0015    0.0016
    0.0009    0.0001    0.0003    0.0012    0.0007    0.0015    0.0037    0.0011
    0.0023    0.0002    0.0012    0.0011    0.0029    0.0016    0.0011    0.0046

>> c

c =

     0
     0
     0
     0
     0
     0
     0
```

```
     0

>> for a=1:length(goal_R)
A=-mu; b=-goal_R(a); Aeq=[ones(1,n)]; beq=[1];
[x(a,:), fval(a,1)] = quadprog(Q, c, A,b, Aeq,beq, [],[]);
std_devi(a,1)=(x(a,:)*Q*x(a,:)')^.5;  % standard deviation = (x'*Q*x)^.5
end

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.
```

```
Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>
>> plot(std_devi, goal_R, '-k*')
xlabel('volatility \sigma')
ylabel('expected return goal R')
title('The efficient frontier of MVO')
>>
>> x

x =

    0.0677    0.8944    0.0292    0.0411   -0.0294    0.0277    0.0168   -0.0475
    0.1091    0.8575    0.0066    0.0474   -0.0198    0.0322    0.0234   -0.0565
    0.1746    0.7993   -0.0291    0.0572   -0.0046    0.0393    0.0338   -0.0706
    0.2401    0.7410   -0.0648    0.0671    0.0107    0.0464    0.0443   -0.0847
```

```
    0.3055    0.6826   -0.1005    0.0769    0.0259    0.0536    0.0547   -0.0988
    0.3710    0.6243   -0.1362    0.0868    0.0411    0.0607    0.0651   -0.1129
    0.4365    0.5660   -0.1719    0.0966    0.0564    0.0678    0.0755   -0.1270
    0.5020    0.5077   -0.2076    0.1065    0.0716    0.0749    0.0859   -0.1411
    0.5675    0.4494   -0.2433    0.1163    0.0868    0.0821    0.0964   -0.1553
    0.6330    0.3911   -0.2789    0.1262    0.1021    0.0892    0.1068   -0.1694

>> std_devi

std_devi =

    0.0139
    0.0140
    0.0148
    0.0161
    0.0179
    0.0201
    0.0224
    0.0249
    0.0276
    0.0303

>> 75264300.08-0.0
```